

Synchra: A Privacy-Preserving Programmable Leverage and Yield Layer for Arc

Synchra Protocol Contributors

<https://synchra.org>

Abstract

Circle’s Arc blockchain provides institutional-grade settlement infrastructure – deterministic sub-second finality, USDC-native gas, and EVM compatibility – and is explicitly designed as a foundational layer for stablecoin finance, programmable FX, tokenized assets, and agent-driven treasury operations. What Arc does not yet provide is a shared market infrastructure that turns settlement into capital-efficient markets: a unified clearing kernel where USDC, EURC, and yield-bearing collateral such as USYC can back FX hedges, tokenized-asset positions, leveraged strategies, and autonomous agent portfolios under a single portfolio risk framework.

We present **Synchra**, a privacy-preserving, programmable leverage and yield layer purpose-built for Arc. Synchra is differentiated by three defensible moats: **(i) leverage** – portfolio-level margin, collateral reuse across heterogeneous markets, and internal repo/rehypothecation; **(ii) privacy** – encrypted order flow, RFQs, batch auctions, TEE-based matching, and forward compatibility with Arc’s protocol-level privacy roadmap (APS); and **(iii) programmable yield** – USYC-bearing collateral tiers, autonomous strategy vaults, and agent-controlled smart treasuries. The protocol supports perpetual swaps, FX derivatives, real-world asset markets, programmable strategy vaults, and yield-bearing collateral pools within one unified risk and settlement system.

Synchra integrates directly with Circle’s platform – USDC/EURC settlement, USYC yield-bearing collateral, CCTP cross-chain liquidity, Circle’s Agent Stack, Nanopayments, and Developer Controlled Wallets – and is designed to remain composable with Arc’s native privacy roadmap. This paper presents the architecture, formalizes the clearing kernel and portfolio margin model, and analyzes security and economic properties.

Keywords: decentralized finance, portfolio margin, leverage, privacy-preserving execution, yield-bearing collateral, programmable vaults, onchain settlement

Contents

1	Introduction	5
1.1	Thesis	6
1.2	Paper organization	6
2	Background and Motivation	7
2.1	The Arc ecosystem gap	7
2.2	Evolution of decentralized markets	7
2.3	Lessons from clearing-focused architectures	7
2.4	Architectural principles	8
2.5	Limitations of current architectures	9
2.6	Research contribution	9
3	System Model	10
3.1	Actors	10
3.2	Threat model	10
3.3	Design goals	10
4	Synchra Architecture	11
4.1	Architectural philosophy	11
4.2	Layer responsibilities	12
5	Clearing Kernel	12
5.1	Account model	13
5.2	Collateral engine	13
5.3	Position engine	13
5.4	Settlement engine	14
6	Portfolio Margin Engine	14
6.1	Motivation	14
6.2	Margin model	15
6.3	Risk scenario generation	16
6.4	Liquidation mechanism	16
6.5	Insurance fund	17
7	Programmable Market Extensions	17
7.1	Extension interface	17
7.2	Privacy-preserving perpetual module	17
7.3	FX perpetual module	18
7.4	Tokenized RWA and rate module	18
7.5	Strategy vault module	19
7.6	Repo and basis module	19
8	Execution Layer	19
8.1	Public CLOB	19
8.2	Request-for-quote (RFQ)	19
8.3	Batch auctions	20

9	Privacy Architecture	20
9.1	Privacy dimensions	20
9.2	Privacy modes	20
9.3	Private order flow	21
10	Trusted Execution Environment	21
10.1	TEE architecture	21
10.2	TEE security model	21
11	Zero-Knowledge Layer	22
11.1	Solvency proof	22
11.2	Periodic batch verification	22
11.3	ZK for TEE attestation verification	22
11.4	Implementation	22
11.5	Limitations	23
12	Agent-Native Financial Markets	23
12.1	Agent accounts	23
12.2	Agent types	24
12.3	Agent security	24
13	Yield and Strategy Vaults	24
13.1	Yield-bearing collateral tiers	24
13.2	Programmable strategy vaults	25
13.3	Example strategies	25
13.4	Repo and securities lending module	26
13.5	Structured product vaults	26
14	Economic Model	27
14.1	Fee structure	27
14.2	Protocol sustainability	27
14.3	Economic security	28
14.4	Incentive alignment	28
14.5	Value proposition	28
15	Arc Ecosystem Alignment	28
15.1	Alignment with Arc’s philosophy	28
15.2	Filling Arc’s infrastructure gaps	29
15.3	Stablecoin-native settlement	29
15.4	Tokenized collateral via USYC	29
15.5	Nanopayments for agent-to-agent settlement	30
15.6	Agent Stack integration	30
15.7	CCTP interoperability	30
15.8	Restricted product access	30
16	Security Analysis	30
16.1	Oracle manipulation	30
16.2	Liquidation cascades	31
16.3	Smart contract risk	31

16.4 Privacy assumptions	31
16.5 ZK assumptions	31
17 Related Work	31
17.1 Orderbook and liquidity-pool derivatives	32
17.2 Layer separation and specialization	32
17.3 Portfolio margin	32
17.4 Modular exchange architectures	32
17.5 Private execution	32
17.6 Tokenized yield and agent infrastructure	33
18 Implementation Roadmap	33
18.1 Accelerator MVP: privacy-preserving BTC/USDC perpetual on Arc Testnet .	33
18.2 Phase 1: production clearing kernel and cross-market margin	33
18.3 Phase 2: FX, RWA, and strategy vaults	34
18.4 Phase 3: agent-native and privacy-at-scale infrastructure	34
18.5 Stack	34
19 Conclusion	35

1 Introduction

Circle’s Arc blockchain [3] delivers what blockchains were always meant to be: a settlement layer for programmable money with institutional-grade performance. Combining Malachite consensus with EVM compatibility, Arc achieves deterministic sub-second finality, low-latency throughput, and USDC as native gas [2]. Circle describes Arc as the Economic OS for the internet – a stablecoin-native Layer-1 optimized for global payments, programmable FX, tokenized assets, and agent-driven financial workflows.

Yet a settlement chain is not a market. While Arc provides the rails for value movement, it does not yet provide the shared clearing, risk, and execution infrastructure that would allow diverse financial applications to share capital and settle against one another. A lending protocol, an FX hedging venue, and a tokenized-asset derivatives market launching on Arc would each rebuild its own collateral pool, risk engine, and settlement logic. The result is fragmented capital, duplicated fixed costs, and a market layer that develops more slowly than the underlying chain.

The core opportunity is therefore architectural. Modern finance runs on shared clearing infrastructure: central counterparties, prime brokers, and tri-party settlement systems allow collateral to flow across desks, markets, and jurisdictions. In decentralized finance, smart contracts have historically collapsed execution, clearing, and settlement into monolithic applications. This worked for simple token swaps, but it fails for the diverse, interconnected markets that define institutional finance.

Synchra introduces a privacy-preserving, programmable leverage and yield layer for Arc. It separates execution from clearing and settlement, enabling any market primitive to plug into a unified collateral pool and risk engine. Synchra’s competitive position rests on three mutually reinforcing moats:

1. **Leverage through portfolio-level margin.** Synchra computes margin across all of a user’s positions, across all market types. A long EURC/USDC FX hedge can offset a USDC-denominated perpetual position; cross-chain collateral via CCTP increases effective capital without forcing pre-positioning; and an internal repo/securities-lending module enables collateral reuse within a unified risk framework. Leverage here is not merely high multiplier exposure – it is the systematic reuse and optimization of collateral, which increases returns on deposited capital and deepens liquidity for every market plugged into the kernel.
2. **Privacy through encrypted execution.** Institutional participants will not move size onchain if their order flow, positions, and strategies are visible to the mempool or to block proposers. Synchra provides configurable privacy – public CLOBs for transparent markets, encrypted RFQs for block trades, TEE-based matching for confidential order flow, and zero-knowledge solvency attestations – all operating as an application-layer complement to Arc’s protocol-level privacy roadmap.
3. **Programmable yield on productive collateral.** A clearing layer that merely holds collateral is capital inefficient. Synchra treats idle margin as productive capital: USYC and other yield-bearing assets earn yield while simultaneously backing positions, and programmable strategy vaults deploy collateral into delta-neutral, FX carry, funding-arbitrage, and RWA-backed strategies – all margined through the same kernel.

The result is that Synchra turns Arc from a settlement chain into a capital-efficient market infrastructure. Traders, enterprises, asset managers, and autonomous agents can

run multi-strategy, multi-currency, cross-market portfolios from a single collateral pool, with deterministic settlement finality and fees denominated in USDC.

1.1 Thesis

We argue that Arc’s full potential as a financial settlement layer requires a shared clearing, risk, and yield infrastructure that is currently missing. Synchra provides this infrastructure. Rather than launching another monolithic exchange, Synchra separates execution from clearing and settlement, enabling any market primitive to plug into a unified collateral pool, portfolio risk engine, and yield framework. The result is not a single protocol but a platform: a privacy-preserving, capital-efficient leverage and yield layer for stablecoin-native markets on Arc.

Synchra realizes this through:

- **A unified clearing kernel** providing shared accounting, collateral management, settlement, and portfolio-level risk assessment across all markets on Arc.
- **A modular market extension framework** allowing financial primitives – FX derivatives, perpetuals, RWA markets, strategy vaults – to be deployed as kernel extensions without replicating infrastructure.
- **A portfolio margin engine** evaluating correlated positions across heterogeneous markets, enabling collateral reuse and systematic hedging credits impossible under isolated margin.
- **Configurable privacy** through encrypted order flow, RFQs, batch auctions, and TEE-based matching – an application-layer privacy layer that is composable with Arc’s protocol-level privacy roadmap (APS).
- **Programmable yield infrastructure** including USYC-bearing collateral tiers, strategy vaults, internal repo markets, and agent-controlled smart treasuries.
- **Agent-native participation** with programmable agent accounts, session policies, and direct integration with Circle’s agent and payment infrastructure.
- **Stablecoin-native settlement** via USDC and EURC, with USYC and other yield-bearing assets as collateral, and CCTP for cross-chain liquidity.

1.2 Paper organization

Section 2 motivates the design through analysis of existing systems and the Arc ecosystem gap. Section 3 formalizes the system model, actors, and design goals. Section 4 presents the layered architecture. Sections 5 through 9 detail the clearing kernel, portfolio margin, programmable markets, execution layer, and privacy architecture. Sections 12, 13, and 14 discuss agent-native markets, yield and strategy vaults, and economic security. Section 15 establishes how Synchra integrates with and complements the Arc ecosystem. Section 17 surveys related work, and Section 18 outlines the implementation roadmap.

2 Background and Motivation

2.1 The Arc ecosystem gap

Arc delivers the technical foundation for institutional onchain finance: Malachite consensus for deterministic sub-second finality, a Reth-based EVM for full EVM compatibility, USDC as native gas for stable-denominated fees, and native integration with Circle’s broader platform including USDC, EURC, CCTP, and USYC [3, 2]. Circle’s broader platform adds tokenized money-market instruments (USYC), Nanopayments for gas-free microtransfers, the Agent Stack for autonomous financial agents, and CCTP bridging across chains.

Arc’s public messaging explicitly targets stablecoin-native use cases: global payments, programmable FX, tokenized assets, and agent-driven treasury operations [3]. The ecosystem requires high-quality tokenized RWAs composable with DeFi primitives, institutional-grade FX infrastructure, and smart treasury agents. What is not on Arc’s public roadmap is the shared clearing, risk, and yield layer that would allow these applications to share collateral and settle against one another. That is Synchra’s opportunity.

2.2 Evolution of decentralized markets

DeFi derivatives have progressed through several architectural generations:

First generation (2020–2021) established AMM-based perpetuals. These used virtual AMMs for price discovery, inheriting liquidity limitations from spot AMMs.

Second generation (2021–2023) introduced hybrid models. Liquidity-pool protocols used pooled collateral as counterparty, enabling deep markets without orderbooks but exposing liquidity providers to concentrated and often asymmetric risk.

Third generation (2023–2025) brought orderbook-based derivatives to dedicated execution environments. These environments demonstrated that onchain CLOBs could match centralized exchange latency, and several protocols began introducing portfolio margin concepts within single asset classes.

Concurrent innovations include modular exchange extensibility, private execution via zero-knowledge and TEE-based matching, the emergence of tokenized money-market funds as yield-bearing collateral, and clearing-focused architectures that separate clearing, margin, and execution layers.

2.3 Lessons from clearing-focused architectures

Modern clearing-focused exchange architectures validate several principles that Synchra adopts:

- **Layer separation.** Clearing, risk, and execution should be independent components with clean interfaces.
- **Account-level cross margin.** Margin requirements should be computed at the account level rather than per-position.
- **Multi-tier liquidation cascade.** Standard liquidation, backstop liquidation, and auto-deleveraging provide a robust default waterfall.
- **Per-second funding accrual.** Cadence-invariant funding simplifies position accounting.

Synchra applies these principles specifically for Arc. Arc’s deterministic sub-second finality removes the need for custom consensus clusters or geographic colocation. Synchra’s moats are leverage through cross-market portfolio margin, privacy through encrypted execution, and programmable yield through yield-bearing collateral and strategy vaults. Synchra is designed for the stablecoin-native, agent-driven market layer Arc is building toward.

2.4 Architectural principles

Our design is shaped by several hard-won lessons from the evolution of on-chain trading infrastructure:

1. **Separate concerns.** Clearing, risk, and execution should be independent systems with narrow interfaces. This allows each layer to be optimized, scaled, and audited independently, and makes the protocol robust to changes in any one layer.
2. **Shard by account and market.** Most financial operations touch only two parties and one market. Designing state around accounts and markets enables parallel execution and horizontal scaling without global coordination.
3. **On-chain settlement truth, off-chain latency.** Final collateral balances, positions, and obligations must be verifiable on-chain. Off-chain components should only handle sequencing, privacy, or computation that can be independently verified.
4. **Gas-aware market design.** Gas cost is a market-structure variable, not an implementation detail. Hot paths such as post, cancel, and modify must be storage-efficient so that market makers can quote tight spreads.
5. **Cadence-invariant economics.** Funding, yield, and fee accrual should be continuous functions of time and state, so that a participant’s PnL does not depend on when a settlement event happens.
6. **Staged default waterfall.** Insolvency should escalate through a predictable ladder: standard liquidation, backstop liquidation, auto-deleveraging, and finally contract unwind. Clarity prevents panic-driven cascades.
7. **Oracle-tethered mark pricing.** A derivative’s mark price should be anchored tightly to the underlying spot price through multiple independent feeds, with impact-based pricing to resist manipulation.
8. **Permissionless safety functions.** Liquidation, funding settlement, and proof verification should not require privileged operators. Anyone must be able to enforce protocol solvency.
9. **Economic cleanup incentives.** Markets produce garbage: expired orders, stale quotes, dust positions. The protocol should pay participants to remove them.
10. **Compartmentalized risk with global backstops.** Risk should be isolated by market and vault so that one failure does not contaminate the whole system, but a global backstop should exist for tail events.
11. **Colocate with the economic center.** Execution infrastructure should sit as close as possible to the source of economic activity. For Synchra on Arc, this means native integration with USDC, EURC, USYC, CCTP, Circle’s Agent Stack, and Nanopayments – not treating Arc as one of many supported chains.

2.5 Limitations of current architectures

Despite these advances, all existing systems share fundamental constraints:

Exchange-centric architecture. Every protocol is built as a standalone exchange. This requires each to bootstrap liquidity, develop a risk engine, and manage settlement independently. The economics of marginal benefit and high fixed cost create natural monopolies.

Single-market risk. Margin is computed per-position or per-market. A trader with offsetting positions across protocols receives no cross-margin benefit. Even within protocols supporting multiple markets, risk aggregation is typically additive rather than portfolio-based.

Transparency as default. Onchain execution reveals order parameters – price, size, side, leverage – to the mempool or block proposer before execution. This enables MEV extraction estimated at hundreds of millions annually [8] and deters institutional flow.

Closed extension models. Adding a new market type requires forking and redeploying. There is no standard interface for plugging new financial primitives into existing risk and clearing infrastructure.

Capital isolation. Collateral deposited in one protocol is inaccessible to others, even when the same asset is used. This fragments what should be a single pool of systemic collateral.

2.6 Research contribution

Synchra’s contribution is an architectural synthesis that advances the state of onchain financial infrastructure while aligning closely with Arc’s stated priorities. The specific contributions are:

1. **A clearing kernel for Arc.** Synchra provides a shared clearing, risk, and collateral infrastructure for capital markets on Arc – deployed as modular extensions rather than isolated protocols.
2. **Clearing kernel formalization.** We define a minimal interface for onchain clearing that separates market logic from risk, collateral, and settlement, and argue that any market satisfying the kernel interface can share a unified collateral pool without compromising kernel invariants. A full invariant proof is left to a companion specification document accompanying the reference implementation.
3. **Portfolio margin across heterogeneous markets.** We extend portfolio-based margin (present in traditional futures clearing houses and in some single-asset-class onchain protocols) to a heterogeneous market model supporting perpetuals, FX derivatives, RWA markets, and strategy vaults simultaneously within one collateral pool.
4. **Privacy-preserving execution.** We design a configurable privacy architecture that spans encrypted order flow, RFQs, batch auctions, TEE-based matching, and ZK solvency attestation, without requiring full ZK matching for every order.
5. **Programmable yield framework.** We integrate yield-bearing collateral tiers, internal repo, and strategy vaults into the clearing kernel so that collateral can earn yield while backing positions.
6. **Agent-native design.** We specify autonomous agents as first-class clearing participants with programmable permissions, session keys, and direct integration with Circle’s Agent Stack and Nanopayments.

3 System Model

3.1 Actors

Synchra defines five participant types:

Traders Human or automated participants who open, modify, and close positions. Traders interact through the execution layer and maintain accounts in the clearing kernel.

Institutions Organizations with multi-user access control, compliance requirements, and portfolio-level risk management. Institutions have hierarchical account structures with sub-account isolation.

Agents Autonomous programs registered as protocol participants with cryptographic identities, programmable permissions, and execution policies. Agents are first-class accounts, not scripts operated by traders.

Liquidity providers Participants who deposit collateral into liquidity pools for specific markets, earning fees and funding payments in exchange for assuming inventory risk.

Validators/Sequencers Infrastructure operators responsible for transaction ordering, state transitions, and in the case of TEE execution, attestation of confidential computations.

3.2 Threat model

Synchra assumes the following threat landscape:

- **Consensus layer security** is provided by the underlying chain (Arc EVM). We assume consensus is Byzantine fault tolerant.
- **Smart contract risk** is managed through formal verification of the clearing kernel and independent audits of market extensions.
- **Oracle manipulation** is mitigated through redundant oracle feeds, TWAP-based pricing, and circuit breakers.
- **MEV and front-running** are addressed through encrypted order flow, batch auctions, and fair ordering within the execution layer.
- **TEE trust** requires trust in hardware manufacturers (Intel TDX/AMD SEV-SNP). We treat TEE compromise as low-probability but provide ZK fallback for verification.
- **Privacy leakage** through side channels (metadata, timing, network analysis) is acknowledged as a limitation of the current architecture.

3.3 Design goals

G1 — Unified liquidity. Collateral deposited in the clearing kernel must be available to support positions in any market extension, subject to portfolio-level risk constraints.

G2 — Portfolio risk. Margin requirements must be computed holistically across all of a trader’s positions, incorporating correlation, hedging, concentration, and liquidity factors.

- G3 — Modular markets.** Any financial primitive satisfying the kernel’s market extension interface must be deployable without modifying the core clearing logic.
- G4 — Leverage and capital efficiency.** The system must maximize collateral reuse – yield-bearing collateral, cross-market hedging credits, and cross-chain collateral – while controlling default risk.
- G5 — Configurable privacy.** Users must control the confidentiality level of each order and position, from fully public to encrypted/TEE-protected execution with selective disclosure.
- G6 — Agent compatibility.** Autonomous agents must be native protocol participants with programmable permissions, cryptographic identities, and execution policies.
- G7 — Deterministic settlement.** Settlement must be deterministic and independently verifiable from execution, ensuring that the clearing kernel remains the authoritative source of financial truth.
- G8 — Yield-bearing collateral.** Collateral must generate yield while backing positions, so that capital efficiency includes passive income as well as leverage.

4 Synchron Architecture

Synchron is organized as a layered architecture, where each layer abstracts a distinct concern and communicates through well-defined interfaces.

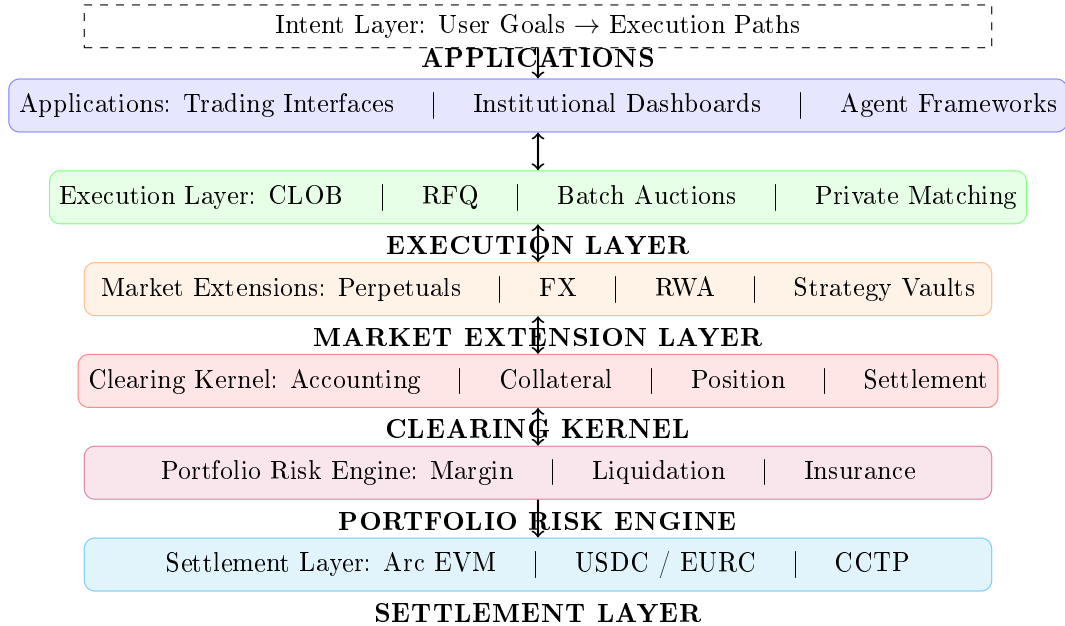


Figure 1: Synchron layered architecture. Arrows indicate bidirectional communication except between the risk engine and settlement layer, where settlement is downstream.

4.1 Architectural philosophy

The architecture follows three principles:

1. **Separation of concerns.** Each layer is independently upgradeable and testable. Market extensions cannot compromise the clearing kernel’s invariants.
2. **Minimal core, maximal extension.** The clearing kernel implements only what is universally required: accounting, collateral management, and settlement. Market-specific logic lives in extensions.
3. **Deterministic settlement.** The clearing kernel’s state transitions are deterministic functions of authenticated market outputs. Execution may involve off-chain computation (matching, risk evaluation), but settlement always reduces to onchain state transitions.

4.2 Layer responsibilities

Application layer. Frontends, dashboards, and agent frameworks that translate user intent into protocol actions. The application layer communicates through the intent layer, which abstracts execution path selection.

Execution layer. Handles order matching, price discovery, and trade execution. Supports multiple execution modes: a public CLOB for transparent markets, an RFQ system for block trades, batch auctions for fairness, and a private matching engine for confidential orders.

Market extension layer. Contains the financial logic for specific market types. Each extension implements the ‘IMarket’ interface defined by the clearing kernel and manages market-specific state: funding rates, leverage parameters, oracle feeds, and resolution mechanisms.

Clearing kernel. The core financial state machine: account abstraction, collateral vault, position tracking, and settlement engine. The kernel is market-agnostic – it manages balances and positions without understanding their semantics.

Portfolio risk engine. Computes collateral requirements across all of a trader’s positions. Runs liquidations when requirements are breached. Manages the insurance fund. The risk engine is the only component with a global view of trader positions.

Settlement layer. The underlying chain infrastructure. Synchronra is designed for Arc EVM, leveraging native USDC/EURC settlement, CCTP for cross-chain transfers, and Arc’s low-latency finality.

Each layer communicates through a narrow interface. The execution layer does not hold collateral; the risk engine does not match orders; the clearing kernel does not price markets. This separation is the foundation of Synchronra’s modularity and security.

5 Clearing Kernel

The clearing kernel is the central state machine of Synchronra. It maintains the authoritative record of accounts, collateral, positions, and obligations. All market extensions settle through the kernel, ensuring that settlement is unified even when execution is distributed.

The kernel is intentionally minimal. It does not understand market semantics; it understands debits, credits, and the interface through which extensions request changes to account state. This minimalism keeps the trusted computing base small and makes formal verification tractable.

5.1 Account model

Each participant has a **clearing account** that aggregates:

- **Collateral balance** by asset type (USDC, EURC, USYC, and accepted tokenized assets).
- **Position set** across all market extensions, keyed by market ID.
- **Credit record** for funding payments, fees, and liquidation penalties.

Formally, an account A is a tuple:

$$A = (\mathcal{B}, \mathcal{P}, \mathcal{H})$$

where $\mathcal{B}$ is a vector of collateral balances $\{b_a \mid a \in \mathcal{A}\}$, $\mathcal{P}$ is a set of positions $\{p_m \mid m \in \mathcal{M}\}$ indexed by market, and $\mathcal{H}$ is a credit/debit history.

5.2 Collateral engine

The collateral engine manages deposits, withdrawals, and valuations. Assets are classified by their **collateral tier**, determining haircuts and eligibility. This four-tier classification (Table 1) is the single authoritative tier taxonomy used throughout this paper; Section 13 describes how each tier accrues yield, and treats cross-chain origin as a haircut modifier applied on top of a tier rather than a tier of its own.

Tier	Assets	Haircut
1	USDC, EURC	0%
2	USYC, stUSD	2%–5%
3	ETH, BTC (liquid staking)	10%–20%
4	Tokenized RWAs	15%–35%

Table 1: Collateral tiers and haircuts. Haircuts are dynamic and adjusted by the risk engine based on volatility and liquidity conditions. An additional cross-chain modifier (typically +2%–5% haircut) applies to any tier-eligible asset bridged via CCTP rather than natively held on Arc, reflecting bridging and rebalancing latency.

The effective collateral value for an account is:

$$\mathcal{C}_{\text{eff}} = \sum_{a \in \mathcal{A}} b_a \cdot (1 - h_a) \cdot p_a$$

where b_a is the balance, h_a the haircut (inclusive of any cross-chain modifier), and p_a the oracle price of asset a .

5.3 Position engine

The position engine maintains the position set $\mathcal{P}$ for each account. A position p_m in market m is defined by extension-specific data but must expose a canonical interface:

The kernel stores positions as opaque bytes with typed accessors, allowing the kernel to remain market-agnostic while the risk engine reads the data it needs.

The **leverage** field on **MarketPosition** denotes the maximum leverage *ceiling* permitted for that position under its market extension’s risk-tier configuration – it is a per-market

Figure 2: IMarket interface (simplified)

```
struct MarketPosition:
  marketId: bytes32
  size: int256 // Positive = long, negative = short
  entryPrice: uint256
  margin: uint256
  leverage: uint256 // Per-market risk-tier ceiling, not effective
  leverage
  function getPnL(oraclePrice) → int256
  function getFunding(position, period) → int256
  function validateOpen(size, collateral) → bool
  function validateClose(position) → bool
```

bound, not the trader’s effective leverage. Because Synchra margins at the portfolio level (Section 6), effective leverage is an emergent, account-level quantity computed by the portfolio risk engine as total notional exposure divided by effective collateral $\mathcal{C}_{\text{eff}}$, and cannot be read off a single position.

5.4 Settlement engine

The settlement engine processes three types of events:

Trade settlement. When a match occurs in the execution layer, the settlement engine validates the matched trade against the risk engine, updates positions, and applies collateral changes atomically. Margin-based instruments (perpetuals, FX derivatives) require both counterparties to post initial margin against their new positions. Instruments that carry an upfront premium transfer the premium from buyer to seller at settlement.

Funding settlement. Funding accrues continuously from a per-market funding index. The settlement engine does not write every position’s funding PnL to storage at every interval; instead, it updates the global funding index and computes a position’s funding PnL lazily when the position changes. This cadence-invariant design ensures that funding payments are independent of the exact block at which settlement is triggered.

Liquidation settlement. When the risk engine flags an account, the settlement engine freezes the account’s positions and delegates liquidation to the appropriate mechanism based on the default waterfall (Section 6).

6 Portfolio Margin Engine

The portfolio margin engine is a principal differentiator of Synchra. Rather than margining each position independently, the engine evaluates the entire portfolio to determine capital requirements.

6.1 Motivation

In existing DeFi derivatives, a trader who is:

- Long 10 BTC notional on Exchange A
- Short 9.5 BTC notional on Exchange B

posts margin on both positions, despite the net exposure being only 0.5 BTC. The gross margin requirement is approximately $(10 + 9.5) \times \text{IM}_{\text{BTC}} = 19.5 \times \text{IM}_{\text{BTC}}$, when the true economic requirement is $0.5 \times \text{IM}_{\text{BTC}}$.

Portfolio margin solves this by computing net risk at the account level.

6.2 Margin model

We define the portfolio margin requirement $\mathcal{M}(\mathcal{P})$ as a function of the position set $\mathcal{P}$:

$$\mathcal{M}(\mathcal{P}) = f_S(\mathcal{P}) + f_C(\mathcal{P}) + f_L(\mathcal{P}) + f_K(\mathcal{P})$$

Where:

- $f_S(\mathcal{P})$ is the **scenario-based margin** component, computed as the maximum loss across a set of price scenarios $\mathcal{S}$:

$$f_S(\mathcal{P}) = \max_{s \in \mathcal{S}} \left(\sum_{p \in \mathcal{P}} V_p(s) - V_p(\text{current}) \right)$$

where $V_p(s)$ is the value of position p under scenario s .

- $f_C(\mathcal{P})$ is the **concentration charge**, penalizing excessive exposure to a single asset or correlated asset group:

$$f_C(\mathcal{P}) = \alpha \cdot \max \left(0, \frac{\max_{g \in \mathcal{G}} \sum_{p \in g} |\text{size}_p|}{\sum_{p \in \mathcal{P}} |\text{size}_p|} - \theta \right)$$

where $\mathcal{G}$ partitions positions by asset group, θ is a concentration threshold, and α is a penalty parameter *expressed in the same currency units as $\mathcal{M}(\mathcal{P})$* (in practice, α is set as a fraction of total portfolio notional), so that the dimensionless ratio term is scaled back into a dollar-denominated margin charge and f_C is dimensionally consistent with f_S , f_L , and f_K .

- $f_L(\mathcal{P})$ is the **liquidity charge**, applied to positions in illiquid markets where liquidation may incur slippage:

$$f_L(\mathcal{P}) = \sum_{p \in \mathcal{P}} |\text{size}_p| \cdot \ell_p$$

where ℓ_p is a market-specific liquidity factor.

- $f_K(\mathcal{P})$ is the **correlation adjustment**, recognizing genuine hedges between positions and penalizing correlated positions that compound directional risk. Let $s_i \in \mathbb{R}$ denote the *signed* size of position i (positive for long, negative for short, matching the `size: int256` field of `MarketPosition`). For two positions p_i, p_j with return correlation ρ_{ij} :

$$f_K(\mathcal{P}) = \beta \cdot \sum_{i \neq j} \rho_{ij} \cdot s_i \cdot s_j$$

where $\beta > 0$ is a hedging credit parameter. Using signed sizes rather than absolute sizes is essential: when two correlated positions ($\rho_{ij} > 0$) are held in *opposite* directions – a genuine hedge – $s_i \cdot s_j < 0$, so f_K is negative and reduces $\mathcal{M}(\mathcal{P})$. When two correlated positions are held in the *same* direction – a concentrated directional bet dressed up as

diversification – $s_i \cdot s_j > 0$, so f_K is positive and *increases* $\mathcal{M}(\mathcal{P})$, since same-direction correlated exposure compounds tail risk rather than offsetting it. An earlier version of this formula used $|s_i| \cdot |s_j|$ with a fixed leading sign, which incorrectly granted a margin discount for any pair of positively correlated positions regardless of direction; the signed formulation above corrects this.

6.3 Risk scenario generation

The scenario set $\mathcal{S}$ is generated dynamically by the risk engine:

- **Shifts:** Parallel shifts of $\pm 5\%$, $\pm 10\%$, $\pm 15\%$ for each tracked asset.
- **Flatteners/Steepeners:** Yield curve scenarios for fixed-income positions.
- **Basis shock:** Funding rate spikes for perpetual positions.
- **Volatility shock:** Implied volatility jumps for options-like positions.
- **Correlation breakdown:** Historical stress periods where correlations diverged.

6.4 Liquidation mechanism

When an account’s portfolio margin $\mathcal{M}(\mathcal{P})$ exceeds its effective collateral $\mathcal{C}_{\text{eff}}$, the account enters a liquidation state.

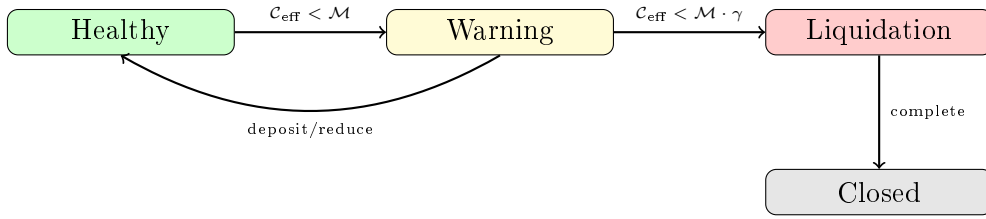


Figure 3: Account liquidation state machine. γ is a liquidation threshold, typically 0.8–0.9.

The liquidation engine may:

- **Standard liquidation:** Close positions against the public order book at market prices, distributing any forfeitable margin to liquidators and the insurance fund.
- **Backstop liquidation:** If the public book cannot absorb the liquidation without excessive slippage, route the position to a backstop liquidity pool composed of protocol-designated market makers, vaults, or the insurance fund.
- **Auto-deleveraging (ADL):** If the backstop is insufficient and the position would create bad debt, forcibly close the underwater position against profitable opposing positions at the bankruptcy price.
- **Contract unwind:** In an extreme tail event, a market may be paused and unwound pro-rata across participants.

This staged waterfall gives participants predictability, protects the broader protocol from one market’s failure, and ensures that solvency enforcement can escalate with market stress.

6.5 Insurance fund

The insurance fund provides a buffer against liquidation deficits. It is funded by a portion of trading fees, liquidation penalties from underwater accounts, and optional LP-provided insurance capital earning a premium.

The fund is compartmentalized by market where practical, so that losses in one market do not automatically drain the global backstop. After market-level backstops are exhausted, a global insurance fund may be drawn upon. The global fund size target F^* is determined by:

$$F^* = \kappa \cdot \sqrt{\sum_{p \in \mathcal{P}_{\text{all}}} \text{size}_p^2}$$

where κ is a risk parameter to be calibrated against historical tail risk (Section 14).

7 Programmable Market Extensions

Synchra defines a **market extension framework** that allows any financial primitive to be deployed as a clearing kernel extension. Extensions implement a canonical **IMarket** interface and expose lifecycle callbacks that the kernel invokes around position, funding, and liquidation events. This design adapts the hook-extensibility pattern to Arc’s EVM environment and Synchra’s clearing-kernel model.

The framework is intentionally permissive about execution semantics. An extension may use a public CLOB, an RFQ network, a batch auction, or a TEE-based matcher; the only requirement is that the resulting position changes and cash flows be settled through the kernel. This separation lets execution innovators compete on latency, privacy, and pricing while the kernel guarantees solvency.

7.1 Extension interface

Each market extension must implement the **IMarket** interface (Algorithm 2) and expose the following lifecycle callbacks:

Figure 4: Market extension lifecycle callbacks

```
callback beforeOpenPosition(user, market, size, price)
callback afterOpenPosition(user, market, position)
callback beforeClosePosition(user, market, position)
callback afterClosePosition(user, market, pnl)
callback beforeSettleFunding(market, rate)
callback onLiquidation(user, market, position)
callback onOracleUpdate(market, price)
```

Extensions are stateless with respect to the kernel – they may maintain their own state (e.g., funding accumulators, LP shares) but must deterministically settle through the kernel.

7.2 Privacy-preserving perpetual module

The perpetual module implements standard BTC/ETH perpetuals with configurable privacy:

- **Funding rate:** computed continuously from a per-market funding index, so that funding payments are independent of the exact settlement block:

$$\Delta F_t = \text{clamp} \left(\frac{P_{\text{mark}} - P_{\text{index}}}{P_{\text{index}}}, -\text{max_rate}, +\text{max_rate} \right) \cdot \Delta t$$

where Δt is the time elapsed since the last index update. A position's funding PnL is computed lazily as $Q \cdot (F_{\text{current}} - F_{\text{entry}}) \cdot P_{\text{index}}$.

- **Mark price:** median of external feeds, on-chain impact mid-price, basis-adjusted fair value, and best bid/ask/last trade, clamped tightly to the oracle index price to resist manipulation.
- **Leverage:** 1x–50x market-level ceiling, adjustable by risk tier; effective account leverage is computed by the portfolio risk engine, not read from a single position (Section 5).
- **Liquidation:** Maintenance margin at 50%–75% of initial margin, adjusted by portfolio risk.
- **Private mode:** Takers may route orders through the encrypted orderbook, preventing leakage of size and position direction.

7.3 FX perpetual module

The FX module enables trading of currency pairs:

- **Pairs:** EURC/USDC, GBP/USD, JPY/USD (with appropriate settlement assets).
- **StableFX integration:** FX positions settled in USDC use CCTP-based conversion at settlement time, removing the need for deep EURC/USDC LPs and aligning with Circle's programmable FX vision.
- **Funding:** Interest rate differential-based funding reflecting the carry trade:

$$r_{\text{FX}} = r_{\text{quote}} - r_{\text{base}}$$

- **Hedging utility:** Enterprises receiving EURC can hedge FX exposure without leaving the Arc ecosystem.

7.4 Tokenized RWA and rate module

Real-world asset derivatives and rate exposures:

- **Underlyings:** Tokenized money-market yields (USYC), treasury yields, gold, credit indices.
- **Pricing:** Oracle-driven with redundant feeds and circuit breakers.
- **Settlement:** Cash-settled in USDC – no physical delivery.
- **Yield trading:** Users can trade exposure to USYC yield, SOFR-like rates, or Treasury yields as synthetic instruments, hedging rate risk within the same portfolio.

7.5 Strategy vault module

Strategy vaults are deployed as market extensions that trade through the kernel on behalf of depositors (full details in Section 13). The module exposes:

- **Vault registration:** a vault declares its strategy rule set, allowed markets, leverage bounds, and performance fee.
- **Execution callbacks:** the kernel calls vault-defined position-adjustment logic at re-balance triggers.
- **Risk isolation:** each vault has its own sub-account and portfolio margin, protecting other kernel participants from vault-specific strategy failures.

7.6 Repo and basis module

A repo market extension allows collateralized borrowing and lending within the kernel:

- **Secured loans:** borrowers post eligible collateral and receive USDC/USYC against a haircut.
- **Term and overnight repo:** configurable maturity, with rates set by an order-driven funding market.
- **Integration:** repo loans are liabilities and repo deposits are assets within portfolio margin, enabling basis-trade vaults and treasury-collateralized leverage.

8 Execution Layer

The execution layer provides multiple execution modes to accommodate different market requirements and privacy preferences.

8.1 Public CLOB

The public central-limit-order-book extension supports transparent, liquid markets. It is designed with gas efficiency as a first-class constraint: post, cancel, and modify operations should be storage-light so that market makers can update quotes frequently without paying prohibitive gas. Synchra uses compact, finance-optimized data structures for price-level discovery and order storage, with a single-transaction modify operation that moves an order's storage slot rather than deleting and re-creating it.

Orders are matched by price-time priority. The matching engine validates signatures, maintains the order book, and submits matched trades to the kernel as settlement batches. The matching engine is trusted for sequencing but not for correctness: all fills are independently verifiable against the signed order set.

8.2 Request-for-quote (RFQ)

For block trades and institutional orders, the RFQ system:

1. A taker broadcasts an RFQ specifying market, side, and size (optionally with price limits).
2. Market makers respond with quotes within a time window.

3. The taker selects a quote, and the trade settles atomically.
4. Failed RFQs (no quotes, expired quotes) have no onchain effect.

8.3 Batch auctions

To mitigate MEV, frequent batch auctions (FBAs) are supported:

1. Orders are collected during an accumulation period (typically 1–5 seconds).
2. At the close, a uniform clearing price is computed that maximizes matched volume.
3. All trades execute at the clearing price, eliminating priority gas auction (PGA) advantages.

The clearing price p^* for a batch with bids $\{b_i\}$ and asks $\{a_j\}$ satisfies:

$$p^* = \arg \max_p \min \left(\sum_i \text{vol}_i \cdot \mathbf{1}_{b_i \geq p}, \sum_j \text{vol}_j \cdot \mathbf{1}_{a_j \leq p} \right)$$

9 Privacy Architecture

Synchra provides a layered privacy model where users choose their confidentiality level per order or per position.

9.1 Privacy dimensions

Users may conceal:

1. **Order intent:** The price, size, side, and market of an order are encrypted until execution.
2. **Position size:** The notional exposure of open positions is hidden from other participants.
3. **Leverage:** The leverage multiplier is private.
4. **PnL:** Realized and unrealized profit/loss is hidden.
5. **Portfolio exposure:** The composition of the collateral pool and open positions across markets is concealed.

9.2 Privacy modes

Mode	Visibility	Use case
Public	All order parameters visible	Retail, liquid markets
Protected	Order parameters visible, identity hidden	Whales, institutions
Confidential	Encrypted orders, TEE matching	Large block trades
Selective	Partial disclosure via ZK	Solvency attestation

Table 2: Privacy modes available to Synchra users.

9.3 Private order flow

Confidential orders follow a commit-reveal flow:

1. The trader submits a commitment $H(\text{order}, r)$ where r is a random nonce.
2. At the execution deadline, the trader reveals (order, r) .
3. The execution engine verifies the commitment, matches the order, and settles.
4. If the trader fails to reveal, the order is treated as canceled; the commitment expires.

For continuous execution, orders are submitted directly to the TEE matcher.

10 Trusted Execution Environment

TEEs provide a practical middle ground between fully transparent and fully private execution, offering confidentiality with hardware-level trust.

10.1 TEE architecture

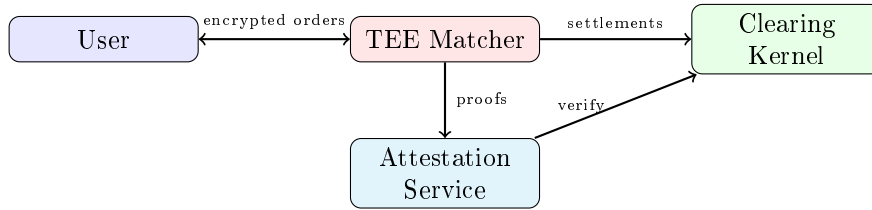


Figure 5: TEE-based execution flow.

Order submission: Users encrypt orders with the TEE’s public key. The TEE decrypts within the enclave, ensuring plaintext never leaves the hardware boundary.

Matching: The TEE matcher processes orders with access to the full order book. Matching logic is identical to the public CLOB but operates on decrypted orders.

Attestation: The TEE produces a signed attestation of each match, proving correct execution within the enclave. Attestation is verified on-chain before settlement.

Fallbacks: If the TEE is compromised (attestation verification fails), the system falls back to on-chain settlement with public execution.

10.2 TEE security model

- **Trust assumption:** The hardware manufacturer (Intel/AMD) is honest. We assume TDX and SEV-SNP provide memory isolation and attestation integrity.
- **Mitigation:** Multiple independent enclave implementations reduce single-vendor risk.
- **Limitation:** Side-channel attacks (cache timing, power analysis) are acknowledged but considered out of scope for the current threat model.

11 Zero-Knowledge Layer

ZK proofs are used selectively for specific high-value operations where proof of correctness is required without revealing the underlying data. We explicitly do not use ZK for per-order matching or execution, as the latency and cost of proof generation make it impractical for high-frequency trading. Instead, ZK is applied where it provides the greatest marginal value: solvency attestation, periodic batch verification, and TEE attestation verification.

11.1 Solvency proof

Institutions and agents can periodically prove solvency without revealing positions:

$$\pi_{\text{solv}} : \text{Prove} \left(\sum_{a \in \mathcal{A}} \text{collateral}_a > \sum_{p \in \mathcal{P}} \text{liability}_p \right)$$

This enables transparent solvency reporting – a requirement for regulated institutions – without portfolio disclosure. The proof is generated off-chain by the institution and verified on-chain, allowing anyone to confirm that an account is solvent without learning its composition or size.

11.2 Periodic batch verification

For non-continuous verification, a matching engine can periodically produce a ZK proof that a batch of settlements was computed correctly:

$$\pi_{\text{batch}} : \text{Prove} (\forall t \in \mathcal{T}_{\text{batch}} : \text{settlement}_t = f(\text{order}_t, \text{state}_t))$$

This proof is generated at discrete intervals (e.g., hourly) rather than per-order, amortizing the proving cost across many trades. The verification contract confirms correctness without re-executing the match logic.

11.3 ZK for TEE attestation verification

Following the approach demonstrated by Kalypso [9], we use ZK proofs to compress TEE attestation verification. Verifying an Intel TDX or AMD SEV-SNP attestation certificate directly on-chain costs approximately 75M gas. By generating a ZK proof that the attestation is valid, this cost is reduced to roughly 300K gas – a factor of 200x improvement. This is applied as a defense-in-depth layer: the TEE provides execution privacy, and the ZK-compressed attestation provides verifiable integrity at practical cost.

11.4 Implementation

For solvency and batch proofs, the ZK suite uses:

- **Backend:** Arkworks (Rust) for constraint generation.
- **Constraint system:** R1CS.
- **Proof system:** Groth16 with BN254 curve.
- **Verification:** Solidity verifier contracts deployed on Arc EVM.

11.5 Limitations

- Groth16 requires a trusted setup per circuit. We use a multi-party ceremony.
- Proof generation cost limits ZK application to low-frequency, high-value operations (solveny attestations, hourly batch verification). For per-order privacy, TEE execution or commit-reveal protocols are preferred.
- Recursive proofs may be used to aggregate multiple solveny attestations into a single proof for scaling.

12 Agent-Native Financial Markets

Synchra treats autonomous agents as first-class protocol participants, not as scripts operated by indirectly authorized traders.

12.1 Agent accounts

An agent account extends the standard clearing account with:

- **Cryptographic identity:** The agent has a native key pair, registered on-chain.
- **Permission model:** Fine-grained permissions encoded as a capability token, signed by the account owner’s registered key over the scoped action fields:

$$\text{capability} = \text{Sign}_{sk_{\text{owner}}}(H(\text{agent_id}, \text{action}, \text{limits}, \text{expiry}))$$

The kernel verifies the signature against the account’s registered owner key before honoring any action authorized under the capability; the hash alone identifies the scoped grant, and the signature is what makes it enforceable and non-forgeable.

- **Execution policy:** Constraints on what the agent can do, enforced at the kernel level:
 - Maximum position size per market.
 - Maximum leverage.
 - Allowed market types.
 - Daily loss limits.
 - Cooldown periods between actions.
- **Session keys:** Ephemeral keys with limited scope and expiry, allowing agents to operate without exposing the master key.
- **Nanopayments integration:** Agents use Circle’s Nanopayments [5] for gas-free sub-cent USDC transfers, enabling high-frequency, low-value transactions such as API payments, streaming data fees, and per-compute-unit billing that would be uneconomical on traditional payment rails.
- **Agent Stack compatibility:** Agent accounts are designed to integrate with Circle’s Agent Stack [4], providing a standard interface for connecting AI models to wallet infrastructure, payment authorization, and onchain settlement.

12.2 Agent types

Trading agents Execute strategies with configurable risk parameters. Trading agents are constrained by the portfolio margin engine – they cannot open positions that would violate the account’s risk limits.

Treasury agents Manage hedging, yield optimization, and FX exposure for institutions. A treasury agent might maintain a delta-neutral BTC position while earning funding, or hedge EURC-denominated liabilities.

Liquidity agents Provide two-sided quotes in RFQ markets. Liquidity agents implement market-making strategies and are compensated through the spread.

Research agents Aggregate information and reflect it in prediction or event-derivative markets. Research agents are the information backbone of the event-derivatives ecosystem.

12.3 Agent security

Agent compromise is mitigated by:

- **Principle of least privilege:** Each agent capability is narrowly scoped.
- **Circuit breakers:** Agents have programmable spending and loss limits.
- **Session expiry:** Keys are ephemeral and must be periodically refreshed.
- **Revocation:** The account owner can revoke any agent capability at any time.
- **Anomaly detection:** Off-chain monitoring systems flag unusual agent behavior.

13 Yield and Strategy Vaults

A clearing layer that merely holds collateral is capital inefficient. Synchra treats idle collateral as productive capital: it earns yield while simultaneously backing positions, and it can be deployed into programmatic strategies that themselves trade through the kernel. This section formalizes the yield-bearing collateral tiers and the programmable strategy vault framework.

13.1 Yield-bearing collateral tiers

Synchra’s collateral tiers are exactly the four tiers introduced in Table 1 (Section 5); this section describes how each tier accrues yield. Cross-chain origin is *not* a separate tier: an asset bridged via CCTP retains its base tier classification and simply carries the cross-chain haircut modifier noted in Table 1.

Tier 1 (USDC, EURC). Instant settlement with Arc’s native stablecoins. Depositors may opt into a sweep module that routes idle Tier-1 balances into overnight repo or money-market instruments, earning a baseline yield while preserving same-block liquidity for margin calls.

Tier 2 (USYC, stUSD). USYC is a tokenized representation of a regulated money-market fund and is already a yield-bearing asset [6]. Held as Tier-2 collateral, USYC contributes its fund yield to the depositor and receives a reduced haircut due to its high credit quality and liquidity.

Tier 3 (ETH, BTC liquid staking). Liquid staking derivatives accrue native staking yield while posted as collateral, subject to the higher haircut reflecting price volatility and slashing risk relative to Tiers 1–2.

Tier 4 (tokenized RWAs). Longer-duration tokenized Treasuries, repo instruments, or credit products earn higher yield but carry higher haircut and liquidity constraints. Tier-4 collateral is marked through independent oracles and subject to stress-based haircuts.

Cross-chain modifier. USDC, EURC, or USYC bridged via CCTP from another chain retains the base-tier treatment above (Tier 1 or Tier 2 respectively) but is subject to the additional haircut increment specified in Table 1, reflecting bridging and rebalancing latency rather than a distinct risk class.

Yield attribution. Each collateral asset accrues its native yield to the depositor’s account, net of any protocol keep fee (typically 0–10 bps). The yield is computed continuously and can be reinvested or withdrawn subject to margin requirements. Importantly, yield accrual does not reduce the collateral’s eligibility for margin; the asset’s value for margin purposes is its market value plus accrued yield.

13.2 Programmable strategy vaults

Strategy vaults are smart-contract-managed sub-accounts that execute predefined strategies through the Synthra kernel. They are first-class participants: each vault has its own account, collateral allocation, risk limits, and position set, all governed by the same clearing kernel.

Vault mechanics. A strategy vault:

1. receives deposits from users in approved collateral assets;
2. allocates capital across one or more market extensions according to a strategy rule set;
3. opens leveraged positions through the kernel, subject to the vault’s own portfolio margin;
4. accrues trading PnL, funding payments, and collateral yield;
5. periodically rebalances and distributes returns to depositors, net of a performance fee.

Because vaults trade through the unified kernel, they benefit from the same cross-market margin and privacy options as individual traders. A single vault can simultaneously run a delta-neutral ETH perpetual basis trade, an FX carry position, and a tokenized-Treasury repo leg – all margined together.

13.3 Example strategies

Delta-neutral market-making vault. Provides liquidity on the public CLOB while hedging directional exposure externally. Earns maker fees plus USYC yield on unused collateral.

Basis trading vault. Long spot / short perp (or vice versa) to capture funding-rate premiums. Cross-market margin allows the spot collateral to back the short perpetual.

FX-hedged RWA carry vault. Holds tokenized Treasuries or credit products while shorting the relevant FX exposure through the FX perpetual module.

Funding-rate arbitrage vault. Shorts an over-funded perpetual and longs an under-funded perpetual on correlated markets, capturing funding-rate differentials with offsetting notional.

Collateral transformation / repo vault. Lends idle Tier-2/Tier-4 collateral into the repo module and borrows USDC against it, earning the repo spread while keeping liquidity available for margin.

Privacy-preserving block-trade vault. Aggregates large-size flow through encrypted RFQs and executes via TEE matching, then clears through the kernel.

Smart treasury vault. An enterprise or agent deposits EURC/USDC receipts and automatically hedges FX and rate exposure via programmable policies, rebalancing as invoices or obligations settle.

13.4 Repo and securities lending module

To further increase capital efficiency, Synthra supports an internal repo market:

- Lenders post idle Tier-2/Tier-4 collateral into a repo pool and earn interest from borrowers.
- Borrowers post eligible collateral and receive a loan of the same or different asset, with haircuts and term structures enforced by the kernel.
- Repo positions are integrated into portfolio margin: a repo loan is a liability; a repo deposit is an asset, both marked at market.
- Liquidation of repo positions follows the same auction mechanism as derivative positions.

This module turns Synthra into a tri-party-style collateral management system: collateral can be borrowed, lent, margined, and yield-bearing within a single risk framework.

13.5 Structured product vaults

Synthra's modular design supports structured products issued as vaults:

- **Principal-protected yield notes:** combine a zero-coupon tokenized Treasury with a small options position.
- **Range accruals:** earn enhanced yield if EURC/USDC trades within a range, using FX perpetuals for delta hedging.
- **Autocallables:** knock-in/knock-out structures backed by perp and RWA positions.

Each structured product is simply a vault with a custom strategy rule set and a defined payoff function. The clearing kernel handles the margin, settlement, and liquidation logic uniformly. These are the instruments referenced in Section 5 as carrying an upfront premium at trade settlement.

14 Economic Model

Synchra has no protocol token. Value accrues to depositors, liquidity providers, strategy vault managers, and the protocol treasury through fee revenue and yield-bearing collateral. This model aligns incentives without introducing speculative token dynamics or regulatory ambiguity.

14.1 Fee structure

Synchra charges fees at multiple points:

Fee type	Rate	Beneficiary
Trading fee (maker)	0%–0.02%	Liquidity providers
Trading fee (taker)	0.03%–0.06%	Liquidity providers, protocol treasury
Funding (perp)	Variable	Long/short transfer
Order recycle fee	Configurable	Participant who removes expired order
Withdrawal fee	0%–0.01%	Protocol treasury
Strategy vault performance fee	10%–20%	Vault manager, protocol treasury
Yield keep fee	0%–10 bps	Protocol treasury
Repo spread	10%–25% of interest	Protocol treasury, insurance fund
Liquidation penalty	1%–5%	Liquidator, insurance fund

Table 3: Synchra fee schedule. Rates are dynamic based on market conditions and volume tiers.

Order recycle fees. Orders submitted with an expiry block must attach a small recycle fee. The fee is refunded if the order fills or the issuer cancels it. If the order expires, the first participant who encounters the expired order during book traversal may claim the recycle fee as compensation for removing stale state. This economic cleanup incentive keeps the order book compact and rewards participants for protocol hygiene.

14.2 Protocol sustainability

Revenue from trading fees, vault performance fees, yield keep fees, and repo spreads funds:

- Insurance fund growth and backstop capital.
- Security audits, formal verification, and bug bounties.
- TEE infrastructure and oracle costs.
- Liquidity incentives during bootstrapping (paid in USDC/EURC, not a token).
- Protocol development and governance operations.

Because Synchra does not issue a token, all incentives are paid in stablecoins or yield-bearing assets. This eliminates the need for inflationary emissions, aligns the protocol with Arc’s stablecoin-native thesis, and simplifies regulatory treatment. The tradeoff is that Synchra cannot bootstrap early liquidity with speculative token emissions the way many competing protocols do; early LP and insurance capital must instead be attracted with real fee yield, USYC-denominated incentive programs funded from the protocol treasury, and design choices (deep cross-market margin, privacy) that are difficult for token-incentivized competitors to replicate quickly.

14.3 Economic security

The protocol’s economic security relies on:

Collateral adequacy. Portfolio margin ensures that collateral requirements reflect actual portfolio risk. Scenario parameters will be calibrated against major historical volatility events (e.g., May 2021, the LUNA collapse, November 2022) as part of the risk engine’s pre-launch validation, with backtesting results published ahead of mainnet deployment.

Liquidation robustness. The multi-stage liquidation process prevents cascading failures. Partial liquidation allows orderly position reduction before forced closure.

Insurance fund. The dynamic fund size target F^* is designed to scale with market risk. Initial calibration will target 99.5% coverage under historically observed price movements, to be validated through backtesting prior to mainnet deployment rather than assumed.

Liquidity provider diversification. Risks are distributed across LPs who have heterogeneous risk preferences and can exit according to their liquidity terms.

14.4 Incentive alignment

LP and insurance capital is subject to minimum lockup periods (configurable per pool, typically 7–30 days) to prevent hot-money flows. Returns are proportional to risk assumed, measured by the VaR contribution of capital to each market. Strategy vault managers earn performance fees only on positive returns, aligning their interests with depositors.

14.5 Value proposition

For depositors, Synchra offers:

- **Yield on collateral:** USYC, repo, and sweep yields accrue while assets back positions.
- **Leverage:** portfolio margin allows more exposure per dollar of collateral than isolated-margin alternatives.
- **Privacy:** encrypted execution protects large order flow and strategy leakage.
- **Composability:** one pool of capital serves FX, perpetuals, RWA, and strategy vaults.

For the Arc ecosystem, Synchra increases capital velocity, deepens liquidity across all market extensions, and provides the institutional-grade clearing infrastructure that complex financial applications require.

15 Arc Ecosystem Alignment

Synchra is designed from the ground up for deployment on Circle’s Arc blockchain [3], leveraging the full Circle developer platform for settlement, collateral, and agent infrastructure.

15.1 Alignment with Arc’s philosophy

Circle describes Arc as the foundation for an open, stablecoin-native financial system. Its public priorities include:

- **Stablecoin-native finance:** USDC and EURC as settlement rails, with USDC as native gas.

- **Programmable FX and cross-border payments:** real-time, programmable currency exchange and settlement.
- **Tokenized real-world assets:** money-market funds, Treasuries, and credit products composable with onchain markets.
- **Autonomous agents and smart treasuries:** AI-driven participants managing payment, hedging, and investment workflows.
- **Institutional-grade trust and compliance:** deterministic finality, permissioned/permissionless options, and privacy-preserving infrastructure.

Synchra directly advances each of these priorities. Its clearing kernel is stablecoin-native; its FX perpetual and cross-currency margin modules make programmable FX economically viable; its USYC and RWA collateral tiers turn tokenized assets into productive margin; its agent accounts and strategy vaults enable smart treasuries; and its privacy architecture provides the confidentiality that institutional flow requires. Synchra does not add generic DeFi to Arc – it builds the specific market infrastructure Arc’s own thesis calls for.

15.2 Filling Arc’s infrastructure gaps

The Arc ecosystem today has strong foundations in swaps, payments, and lending, but every category that requires cross-market coordination, privacy, or composable risk management remains underdeveloped. Synchra is purpose-built to fill these gaps.

The clearing kernel provides the cross-market infrastructure that no individual exchange can justify building. The modular extension framework allows perpetuals, FX derivatives, RWA markets, and programmable strategy vaults to share a unified collateral pool and risk engine on Arc – an architecture that is rare on any chain and absent from the current Arc ecosystem. The privacy modes (Section 9) bring confidential execution to Arc as an application-layer complement to its protocol-level privacy roadmap. The agent-native architecture (Section 12) and yield/strategy vault framework (Section 13) integrate with Circle’s Agent Stack, USYC, and Nanopayments – the products Circle is actively prioritizing for the Arc developer ecosystem.

Synchra does not compete with other Arc protocols. It provides the infrastructure layer that makes their markets more capital-efficient. A lending protocol on Arc using Synchra can margin its lending positions against perpetual positions. A programmable FX protocol on Arc can settle through Synchra’s clearing kernel. Synchra is infrastructure, not isolation.

15.3 Stablecoin-native settlement

Arc’s defining architectural choice – using USDC as its native gas token – aligns directly with Synchra’s settlement model. All collateral, margin, fee, and PnL flows in Synchra are denominated in stablecoins (USDC, EURC), eliminating the dual-volatility problem present in protocols where users must manage both position risk and gas token price risk. Arc’s deterministic sub-second finality enables the real-time settlement that portfolio margin requires, where rapid position updates must be reflected in collateral availability.

15.4 Tokenized collateral via USYC

USYC [7], Circle’s tokenized money market fund, serves as yield-bearing collateral within Synchra’s collateral engine (Tier 2, Section 5). This allows traders to earn yield on idle

collateral while maintaining the ability to open and close positions – a capability not available in protocols limited to non-yield-bearing stablecoin collateral.

15.5 Nanopayments for agent-to-agent settlement

Circle’s Nanopayments [5] enable gas-free sub-cent USDC transfers, which are critical for the agent-native architecture (Section 12). Agents performing high-frequency, low-value transactions – API calls, streaming data purchases, per-compute-unit billing – would be uneconomical at standard gas costs. Nanopayments, powered by Circle Gateway, batch these microtransactions and settle them as aggregated onchain transfers, making agent-to-agent micropayments viable at scale.

15.6 Agent Stack integration

Circle’s Agent Stack [4] provides a standard framework for connecting AI models to wallet infrastructure, payment authorization, and onchain settlement. Synchrona’s agent account model (Section 12) is designed to be compatible with this stack, allowing developers to deploy agents that use Circle’s programmable wallets for custody and Synchrona’s clearing kernel for trading and treasury management, within a unified permission model.

15.7 CCTP interoperability

Circle’s Cross-Chain Transfer Protocol [1] enables USDC transfers across supported blockchains, allowing Synchrona to accept collateral originating from other networks without fragmented liquidity. This is particularly important for institutional participants who may hold USDC across multiple chains and require unified treasury management.

15.8 Restricted product access

Some Arc ecosystem products relevant to Synchrona – StableFX, USYC, and Circle Payments Network – are currently permissioned enterprise products. Synchrona’s architecture accommodates this by treating these integrations as optional enterprise features, with core protocol functionality operating on permissionless primitives (USDC, EURC, Arc EVM).

16 Security Analysis

16.1 Oracle manipulation

Synchrona uses a multi-oracle architecture:

- **Primary oracle:** Pyth Network for real-time price feeds.
- **Secondary oracle:** Chainlink for staleness checks.
- **Tertiary oracle:** TWAP from CLOB trades for circuit breaker comparison.

If the primary and secondary oracles diverge by more than δ (configurable, typically 1%–5%), a circuit breaker pauses market operations until convergence is restored.

For event-derivative and oracle-dependent markets, resolution uses a dispute window with bond-based challenges.

16.2 Liquidation cascades

Portfolio margin can create correlated liquidations during market stress. Mitigations include:

- **Gradual liquidation:** Positions are liquidated in tranches rather than all at once.
- **Liquidation queue:** Liquidations are ordered by time of distress, not all executed simultaneously.
- **Circuit breaker:** If the total liquidation volume exceeds a threshold within a window, trading is paused.

16.3 Smart contract risk

The clearing kernel is designed to be formally verified. Key invariants:

- Collateral cannot be created or destroyed outside of authorized operations.
- Position modifications are atomic with collateral updates.
- Settlement is deterministic given the same input state.
- Market extensions cannot manipulate kernel state outside their authorized interfaces.

16.4 Privacy assumptions

- TEE assumes hardware security. Compromise would reveal encrypted order flow but not alter settlement.
- Commit-reveal assumes the reveal phase is incentivized (bonding).
- Metadata leakage (order frequency, timing, network-level patterns) is acknowledged as out of scope.

16.5 ZK assumptions

- ZK proofs are used only for low-frequency operations (solveny attestation, batch verification, TEE attestation compression), not per-order execution.
- Groth16 requires a trusted setup per circuit. We use a multi-party ceremony.
- Proof generation cost, while non-trivial, is amortized across many trades in batch verification or across reporting periods for solveny attestation.

17 Related Work

Synchra builds on decades of work in derivatives clearing, portfolio margin, private execution, modular exchange design, and tokenized yield infrastructure. This section situates the design without naming individual competing protocols.

17.1 Orderbook and liquidity-pool derivatives

Onchain derivatives have evolved through three broad generations. First-generation protocols used virtual AMMs for price discovery, inheriting liquidity limitations from spot AMMs. Second-generation protocols introduced pooled-collateral models, enabling deep markets without orderbooks but concentrating risk in liquidity providers. Third-generation protocols moved to orderbook-based execution on dedicated infrastructure, demonstrating that onchain CLOBs can match centralized exchange latency when execution and settlement are designed together. Synchrona's clearing kernel is agnostic to the execution model above it: it can settle trades from public CLOBs, RFQs, batch auctions, or TEE-based matching.

17.2 Layer separation and specialization

A persistent lesson from modern exchange design is that clearing, risk, and execution should be separated. Specialized clearing layers can remain market-agnostic while execution layers compete on speed, privacy, and pricing. Some designs push this further by separating matching into geographically distributed zones near the source of spot price discovery. Synchrona adapts the separation principle to Arc: the clearing kernel and margin engine are unified and on-chain, while execution can be public, encrypted, or batched depending on the market. Arc's deterministic sub-second finality removes the need for custom consensus clusters or physical colocation; Synchrona's "colocation" is native integration with Arc's USDC/EURC/USYC settlement, CCTP, and Agent Stack.

17.3 Portfolio margin

Traditional clearing houses developed portfolio margining over decades, with SPAN and STANS as industry-standard models. A few onchain protocols have introduced portfolio margin concepts within single asset classes. Synchrona extends this approach to heterogeneous market types – perpetuals, FX derivatives, RWA markets, and strategy vaults – through a unified clearing kernel.

17.4 Modular exchange architectures

Recent exchange designs have shown the power of modular extensibility: hooks that run custom logic around lifecycle events, separated clearing and execution layers, and per-market configuration. Synchrona adopts this philosophy at the clearing layer: market extensions implement lifecycle callbacks around position, funding, and liquidation events, operating on clearing state rather than swap state.

17.5 Private execution

Private onchain execution has been explored through zero-knowledge proofs, multi-party computation, and trusted execution environments. Each approach involves tradeoffs: ZK provides strong confidentiality guarantees but can introduce latency and cost that limit high-frequency use; TEE-based matching offers lower latency but requires attestation and hardware trust assumptions; batch auctions mitigate MEV without hiding positions. Synchrona takes a layered approach: TEE-based execution for latency-sensitive matching, with ZK reserved for higher-level verification tasks such as solvency attestation, batch correctness, and TEE attestation compression.

17.6 Tokenized yield and agent infrastructure

The emergence of tokenized money-market funds as yield-bearing collateral, gas-free micropayment rails, and agent-oriented wallet infrastructure creates the conditions for autonomous treasury and trading agents. Synchra integrates these primitives directly into its clearing kernel: yield-bearing collateral tiers earn while backing positions, and agent accounts operate with programmable policies, session keys, and micropayment support.

18 Implementation Roadmap

18.1 Accelerator MVP: privacy-preserving BTC/USDC perpetual on Arc Testnet

The initial build targets a focused, high-quality demonstration that validates the three moats and creates the foundation for the full accelerator program:

- **Leverage:** clearing kernel with isolated margin, liquidation waterfall, and a single BTC/USDC perpetual extension.
- **Privacy:** encrypted RFQ flow for size orders with TEE-based quote matching.
- **Programmable yield:** USYC as an eligible collateral tier earning yield while backing positions.

Deliverables:

- Clearing kernel smart contracts (Solidity, Arc EVM, Foundry).
- BTC/USDC perpetual market extension with CLOB matching (Rust off-chain, on-chain settlement).
- Basic portfolio margin risk monitor (Rust off-chain, on-chain enforcement).
- Encrypted RFQ module with TEE quote matching (Intel TDX / Phala / Marlin).
- Frontend (Next.js + TypeScript) using Circle Developer Controlled Wallets or App Kit.
- Arc Testnet deployment with USDC gas and USYC collateral integration.

Milestone: A working privacy-preserving perpetual futures market on Arc Testnet.

Timeline: accelerator program duration.

18.2 Phase 1: production clearing kernel and cross-market margin

- Hardened clearing kernel with formal verification targets for core accounting.
- Portfolio margin engine with scenario-based stress tests.
- Additional perpetual markets (ETH, major crypto assets).
- Multi-asset collateral (USDC, EURC, USYC) with dynamic haircuts.
- Public mainnet deployment on Arc.

Timeline: post-accelerator, Q3–Q4 2026.

18.3 Phase 2: FX, RWA, and strategy vaults

- FX perpetual module (EURC/USDC, GBP/USD, JPY/USD) with interest-rate-differential funding.
- Tokenized RWA and rate module (USYC yield trading, Treasury-rate synthetics).
- Programmable strategy vaults (delta-neutral basis, FX carry, funding arbitrage).
- Repo and basis module for internal collateralized lending.
- Cross-chain collateral via CCTP v2.

Timeline: Q4 2026–Q1 2027.

18.4 Phase 3: agent-native and privacy-at-scale infrastructure

- Agent account system with programmable permissions and session keys.
- Circle Agent Stack and Nanopayments integration.
- Agent strategy vaults (smart treasuries) for enterprise FX and yield management.
- ZK solvency proof layer for institutional attestation.
- ZK-compressed TEE attestation verification.
- Institutional sub-account architecture.
- Migration path to Arc Privacy Sector (APS) when live.

Timeline: Q1 2027+.

18.5 Stack

Component	Technology
Smart contracts	Solidity, Foundry
Execution engine	Rust
Risk engine	Rust
TEE framework	Intel TDX, Rust (Phala, Marlin)
ZK framework (solvency, batch)	Arkworks, R1CS, Groth16, BN254
Agent infrastructure	Circle Agent Stack, Nanopayments
Frontend	TypeScript, React, Next.js
Backend services	TypeScript, Node.js
Settlement	Arc EVM, USDC, EURC, USYC, CCTP

Table 4: Synthra implementation stack.

19 Conclusion

Circle Arc delivers institutional-grade settlement infrastructure. But a settlement chain without markets is infrastructure waiting for a purpose. Today, the Arc ecosystem has swaps and lending primitives, but the categories that define a mature financial system – derivatives, programmable FX, tokenized-asset markets, private execution, cross-market risk management, and agent-native trading – are underdeveloped. Synchra answers by building the missing market infrastructure natively on Arc.

By separating execution from clearing, introducing portfolio-level risk management across heterogeneous markets, and providing a modular market extension framework, Synchra fills the infrastructure gaps in the Arc ecosystem with a single, unified architecture. The clearing kernel provides the cross-market infrastructure that no individual protocol can justify building alone. The portfolio margin engine extends traditional clearing-house risk frameworks to onchain markets – perps, FX derivatives, RWA markets, and strategy vaults sharing one collateral pool with systematic offset benefits, correctly priced through signed, directional correlation adjustments rather than blanket credits for co-movement. The privacy architecture brings confidential execution to Arc as an application-layer complement to its protocol-level privacy roadmap. The yield-bearing collateral tiers and programmable strategy vaults turn idle capital into productive capital. The agent-native model integrates with Circle’s Agent Stack, Nanopayments, and Developer Controlled Wallets, directly executing Circle’s vision for autonomous agent economies on Arc.

The key insight is that financial markets share a common substrate – collateral management, position accounting, settlement, and risk evaluation – that can be abstracted into a shared clearing kernel. Market-specific logic becomes pluggable, allowing any financial primitive to interoperate within the same capital pool. On Arc, where this cross-market infrastructure does not yet exist, the abstraction is not an optimization – it is the difference between having an institutional market layer and not having one.

Synchra is built for the Arc accelerator program: a deep, defensible, stablecoin-native leverage and yield layer whose moats are leverage, privacy, and programmable yield. Circle built the settlement layer. Synchra builds the markets.

Acknowledgements. We thank the Circle Arc and Encode Club communities for the accelerator program context that motivated this work. We thank the broader DeFi research community whose innovations in perpetuals, privacy, and modular architecture directly informed this design.

References

- [1] Circle Internet Financial. Cross-chain transfer protocol (cctp). <https://circle.com/cctp>, 2024.
- [2] Circle Internet Financial. Arc technical architecture: Malachite consensus and reth execution. <https://circle.com/arc/technical>, 2025.
- [3] Circle Internet Financial. Circle arc: Programmable settlement for financial applications. <https://circle.com/arc>, 2025.
- [4] Circle Internet Financial. Agent stack: Building autonomous agents on arc. <https://circle.com/agent-stack>, 2026.

- [5] Circle Internet Financial. Nanopayments: Gas-free sub-cent usdc transfers powered by circle gateway. <https://circle.com/nanopayments>, 2026.
- [6] Circle Internet Financial. Usync: Tokenized money market fund. <https://circle.com/usyc>, 2026.
- [7] Circle Internet Financial. Usync: Tokenized money market fund. <https://circle.com/usyc>, 2026.
- [8] Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. Flash boys 2.0: Frontrunning, transaction reordering, and consensus instability in decentralized exchanges. *arXiv preprint arXiv:1904.05234*, 2019.
- [9] Marlin Protocol. Kalypso: Tee attestation verification via zk proofs on symbiotic. <https://marlin.org/kalypso>, 2026.